

V0088

Advanced Information Security

高级信息安全

Instructor: **Prof. Xiaohong Yu**

Graduate School of Information and Communication Technology,
Ajou University, Korea

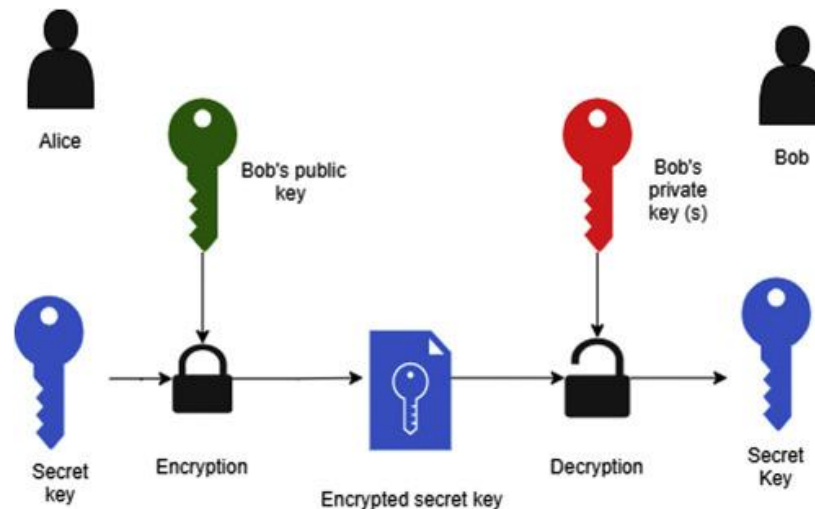
密码学

- 密码学基础 （简介，含义，古典密码，现代密码，以及分类）
- 对称密钥密码 （流密码，分组密码和完整性）
- 公钥密码
- Hash 函数及其他密码
- 高级密码分析

第四章：公钥密码学

公钥密码学

- 两把密钥，一把用于加密，另一把用于解密
 - Alice 使用 Bob 的公钥进行加密
 - 只有鲍勃的私钥才能解密这条消息。



- 基于“活板门，单向功能”
 - “单向”是指在一个方向上容易计算，但在另一个方向上难以计算。
 - 例如：已知 p 和 q ，乘积 $N = pq$ 很容易计算，但很难找到 p 和 q 。来自 N
 - 在创建密钥对时会使用“活门”技术。

公钥密码学

- 对称密钥密码
 - 原文: 通常用P表示, plaintext.
 - 密文: 用C表示
- 公钥密码
 - 原文: 用M表示
 - 密文用C表示

公钥密码学

- 加密

- 假设我们用Bob的公钥对原文M**进行加密** 形成密文C。
- Bob的私钥可以**解密** C恢复M。

- 数字签名

- 鲍勃使用他的私钥进行“加密”**签名**。
- 任何人都可以使用 Bob 的公钥“解密”来**验证签名**。
- 但只有鲍勃才能签名。
- 就像手写签名一样，但更好.....

公钥密码学

- 背包密码
- RSA算法
- Diffie-Hellman 密钥交换
- 椭圆曲线密码 (ECC)

背包密码 (Knapsack)

- 给定一组 n 个权重 $W_0, W_1, \dots, W_{n-1}$ 和一个和 S , 求 $a_i \in \{0, 1\}$ 以便

$$S = a_0 W_0 + a_1 W_1 + \dots + a_{n-1} W_{n-1}$$

(严格来说, 这是子集求和问题)

- **例子**

- 权重 (62、93、26、52、166、48、91、141)
- 问题: 找到一个子集, 使得 $S = 302$
- 答案: $62 + 26 + 166 + 48 = 302$
- 一般背包问题是 NP 完全问题。

背包密码

- 一般背包问题 (GK) 很难解决 (General knapsack)
- 但**超增背包** (SIK) 很容易实现。 (Super Increasing knapsack)
- SIK——每个权重都大于之前所有权重的总和
- 例子
 - 权重 (2、3、7、14、30、57、120、251)
 - 问题：找出元素之和为 $S = 186$ 的子集
 - 从最大重量到最小重量依次进行。
 - 答案： $120 + 57 + 7 + 2 = 186$

背包密码

1. 生成超递增背包 (SIK) 问题
2. 将 SIK 转换为“通用”背包 (GK)问题
3. 公钥: 普通背包问题
4. 私钥: SIK 和转换因子

□ 目标...

- 易于用公钥(GK)加密
- 使用私钥, 易于解密(解决 SIK 问题)
- 如果没有私钥, Trudy 别无选择, 只能尝试解决 GK 问题(这是一个难题)。

背包密码-例子

- 首先 (2,3,7,14,30,57,120,251) 作为 SIK
- 选择 $m = 41$ 和 $n = 491$ (m 和 n 互质, n 超过 SIK 中元素的总和)
- 计算“一般”背包

$$2 * 41 \bmod 491 = 82$$

$$3 * 41 \bmod 491 = 123$$

$$7 * 41 \bmod 491 = 287$$

$$14 * 41 \bmod 491 = 83$$

$$30 * 41 \bmod 491 = 248$$

$$57 * 41 \bmod 491 = 373$$

$$120 * 41 \bmod 491 = 10$$

$$251 * 41 \bmod 491 = 471$$

- “通用”背包: (82,123,287,83,248,373,10,471)

背包密码-例子

- 私钥: $(2, 3, 7, 14, 30, 57, 120, 251)$
 $m^{-1} \bmod n = 41^{-1} \bmod 491 = 12$
- 公钥: $(82, 123, 287, 83, 248, 373, 10, 471)$, $n = 491$
- 示例: Alice 加密信息 $M=150$, 转化成二进制 $M = 10010110$
 $C = 82 + 83 + 373 + 10 = 548$
- 要解密, 请使用私钥.....
 - $548 \cdot 12 = 193 \bmod 491$
 - 求解 (简单) SIK 方程, 其中 $S = 193$
 - 获取明文 10010110

背包密码-例子

- 示例: Alice 加密信息 $M=150$, 转化成二进制 $M = 10010110$

$$C = 82 + 83 + 373 + 10 = 548$$
- 要解密, 请使用私钥.....
- 私钥: $m^{-1} \bmod n$ 和递增背包 (2,3,7,14,30,57,120,251)
- $548 * 12 \bmod 491 = 193$
- 对193解递增背包问题:
- $548m^{-1} = 82m^{-1} + 83m^{-1} + 373m^{-1} + 10m^{-1} = 2mm^{-1} + 14mm^{-1} + 57mm^{-1} + 120mm^{-1} = 2 + 14 + 57 + 120 = 193 \bmod 491$
- 获取明文10010110 $\begin{array}{cccccccc} (2 & 3 & 7 & 14 & 30 & 57 & 120 & 251) \\ (1 & 0 & 0 & 1 & 0 & 1 & 1 & 0) \end{array}$

背包密码-例子

- $m^{-1} \bmod n = 41^{-1} \bmod 491 = 12$

- 这里 m^{-1} 是 m 的模逆。

- 如果存在一个整数 x , 使

$$m * x \equiv 1 \pmod{n}$$

- 那么 x 是 m 在模 n 下的逆元（模逆）记 $m^{-1} \bmod n$ 。

- 这里：

$$41 * 12 = 492 \equiv 1 \pmod{491}$$

- 所以：

$$m^{-1} \bmod n = 41^{-1} \bmod 491 = 12$$

背包密码-弱点

- **陷阱门**：使用模运算将 SIK 转换为“通用”背包
- **单向加密**：普通背包加密容易，破解难；SIK 加密容易破解。
- 这个背包密码系统是**不安全的**。
 - 1983年与Apple II电脑一起损坏
 - 该攻击利用了**格约简**
- “通用背包”还不够通用！
 - 这种特殊的背包很容易损坏

RSA 算法

- GCHQ(英国政府通信总部) 和Rivest、Shamir和Adleman 发明
 - RSA是公钥加密领域的**黄金标准**。
- 设p和q为两个大素数
- 令 $N = pq$ 为**模数**
- 选择与 $(p - 1)(q - 1)$ 互质的
- 求d, 使得 $ed = 1 \bmod (p - 1)(q - 1)$
- **公钥:** (N, e)
- **私钥:** d

RSA 算法

- 消息M被视为一个数字
- 为了加密M，我们计算: $C = M^e \bmod N$
- 解密密文 C，我们计算: $M = C^d \bmod N$
- $M = C^d \bmod N = M^{ed} \bmod N$
- 请注意， e和N是公共的。
- 如果 Trudy 能将 $N = pq$ 分解，她就可以利用e轻松求出d，因为 $ed = 1 \bmod (p-1)(q-1)$ 。
- 因此， **对模量进行因式分解会破解RSA。**
 - 因式分解是破解 RSA 的唯一方法吗？

RSA 算法-真的有效吗?

- 已知 $C = M^e \bmod N$, 我们要证明

$$M = C^d \bmod N = M^{ed} \bmod N$$

- 我们需要用到**欧拉定理**:

如果 x 与 n 互质, 则 $x^{\phi(n)} = 1 \bmod n$

- 事实:

1) $ed = 1 \bmod (p-1)(q-1)$

2) 根据“mod”的定义, $ed = k(p-1)(q-1) + 1$

3) $\phi(N) = (p-1)(q-1)$

- 然后是 $ed - 1 = k(p-1)(q-1) = k\phi(N)$

- 所以, $C^d = M^{ed} = M^{(ed-1)+1} = M \cdot M^{ed-1} = M \cdot M^{k\phi(N)} = M \cdot (M^{\phi(N)})^k \bmod N = M \cdot 1^k \bmod N = M \bmod N$

RSA 算法-示例

- 生成 RSA 密钥对...
 - 选择“大”素数 $p = 11$, $q = 3$
 - 那么 $N = pq = 33$ 且 $(p - 1)(q - 1) = 20$
 - 选择 $e = 3$ (与20 互质)
 - 求 d , 使得 $ed = 1 \bmod 20$
 - 我们发现 $d = 7$ 有效。
- **公钥:** $(N, e) = (33, 3)$
- **私钥:** $d = 7$

RSA 算法-示例

- **公钥:** $(N, e) = (33, 3)$
- **私钥:** $d = 7$
- 假设待加密消息的长度为 $M = 1000 = 8$
- 密文 C 的计算方法如下:
$$C = M^e \bmod N = 8^3 = 512 = 17 \bmod 33$$
- 解密 C 以恢复消息 M
$$\begin{aligned} M &= C^d \bmod N = 17^7 = 410,338,673 = 12,434,505 * 33 + 8 \\ &= 8 \bmod 33 \end{aligned}$$
- 为什么说这是“教科书式”的RSA?

更高效的RSA (1)

- 幂运算示例: $5^{20} \bmod 35$
 - $5^{20} = 95367431640625 = 25 \bmod 35$
- 更好的方法: **反复平方乘**
 - o $20 = 10100$ (以 2 为底)
 - o $(1, 10, 101, 1010, 10100) = (1, 2, 5, 10, 20)$
 - o 注意, $2 = 1 \times 2$, $5 = 2 \times 2 + 1$, $10 = 2 \times 5$, $20 = 2 \times 10$
 - o $5^1 = 5 \bmod 35$
 - o $5^2 = (5^1)^2 = 5^2 = 25 \bmod 35$
 - o $5^5 = (5^2)^2 \cdot 5^1 = 25^2 \cdot 5 = 3125 = 10 \bmod 35$
 - o $5^{10} = (5^5)^2 = 10^2 = 100 = 30 \bmod 35$
 - o $5^{20} = (5^{10})^2 = 30^2 = 900 = 25 \bmod 35$
- 数字不大, 效率更高!

更高效的RSA (2)

- 使用 $e = 3$ (但 N 或 d 可以不同)
 - + 公钥操作仅需 2 次乘法
 - 私钥操作仍然成本高昂。
 - 如果 $M < N^{1/3}$, 则 $C = M^e = M^3$ **立方根攻击**
 - 对于任意 M , 如果将 $C1$ 、 $C2$ 、 $C3$ 发送给 3 个用户, 则立方根攻击有效 (利用中国剩余定理)。
- 可以通过在消息中填充随机比特来防止立方根攻击
- 注: $e = 2^{16} + 1$ 比 $e = 3$ “更好”。

迪菲-赫尔曼算法 (Diffie-Hellman)

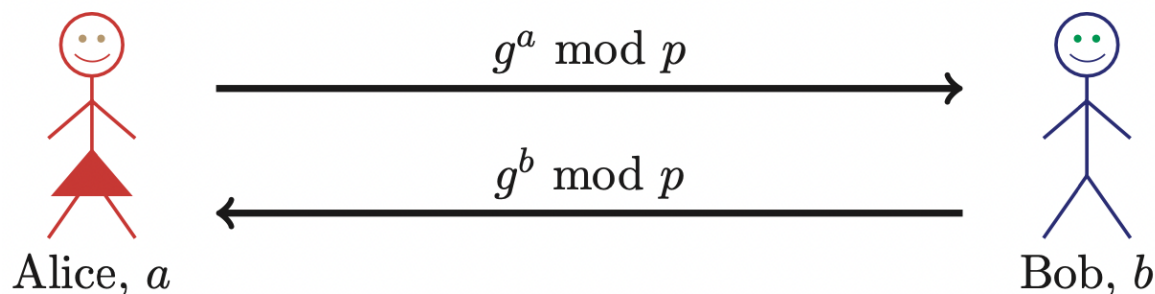
- DH算法 由 GCHQ (英国政府通信总部) 的 Malcolm Williams 发明, 之后, Diffie 和 Hellman 也独立提出该算法。
- “密钥交换”算法
 - 用于建立共享对称密钥
 - 不是用于加密或签名
- DH安全性建立在离散对数问题的计算困难上。
 - 已知: g 、 p 和 $g^k \bmod p$
 - 求: 指数 k

迪菲-赫尔曼算法 (Diffie-Hellman)

- 设 p 为素数, g 为生成元
 - 对于任意 $x \in \{1, 2, \dots, p-1\}$ 有 n 使得 $x = g^n \bmod p$
- 爱丽丝选择她的秘密指数 a 。
- 鲍勃选择他的秘密指数 b 。
- Alice 向 Bob 发送了一个 $g^a \bmod p$ 。
- Bob 向 Alice 发送了一个 $g^b \bmod p$ 。
- 两者都计算共享秘密, $g^{ab} \bmod p$
- 共享密钥可以用作对称密钥

迪菲-赫尔曼算法 (Diffie-Hellman)

- 公钥: g 和 p
- 私钥: Alice 的指数 a , Bob 的指数 b



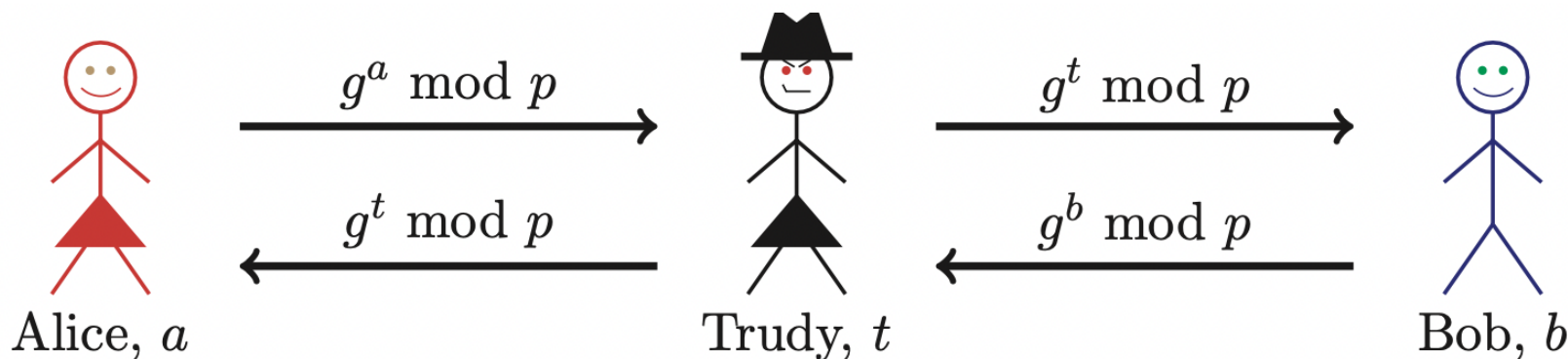
- Alice 计算 $(g^b)^a = g^{ba} = g^{ab} \bmod p$
- Bob 计算 $(g^a)^b = g^{ab} \bmod p$
- 他们可以使用 $K = g^{ab} \bmod p$ 作为对称密钥

迪菲-赫尔曼算法 (Diffie-Hellman)

- 假设 Bob 和 Alice 使用Diffie -Hellman 密钥交换算法确定对称密钥 $K = g^{ab} \bmod p$
- Trudy 可以看到 g 、 p 、 $g^a \bmod p$ 和 $g^b \bmod p$ 。
 - 但是..... $g^a g^b \bmod p = g^{a+b} \bmod p \neq g^{ab} \bmod p$
- 如果Trudy能找到 a 或 b ， 她就能得到 K 。
- 如果 Trudy 能解决**离散对数**问题， 她就能找到 a 或 b 。

迪菲-赫尔曼算法 (Diffie-Hellman)

- 易受中间人攻击



- ❑ Trudy与 Alice分享了秘密 $g^{at} \bmod p$
- ❑ Trudy与 Bob分享了秘密 $g^{bt} \bmod p$
- ❑ 爱丽丝和鲍勃不知道特鲁迪是MiM。

迪菲-赫尔曼算法 (**Diffie-Hellman**)

- 如何防范中间人攻击?
 - 使用对称密钥加密 DH
 - 使用公钥加密 DH 交易
 - 使用私钥对 DH 值进行签名
 - 其他?

椭圆曲线密码学 (ECC)

- “椭圆曲线”**不是**一种密码系统
- 椭圆曲线为公钥系统中的数学运算提供了一种不同的方法。
- DH、RSA 等的椭圆曲线版本
- 椭圆曲线效率更高
 - 用更少的比特就能达到相同的安全性.....
 - 但操作更为复杂。
 - 总的来说，这是一次巨大的“胜利”。

什么是椭圆曲线?

- 椭圆曲线E是形如 的方程的图像。

$$y^2 = x^3 + ax + b$$

- 还包括一个“无穷远点”。

- 考虑椭圆曲线

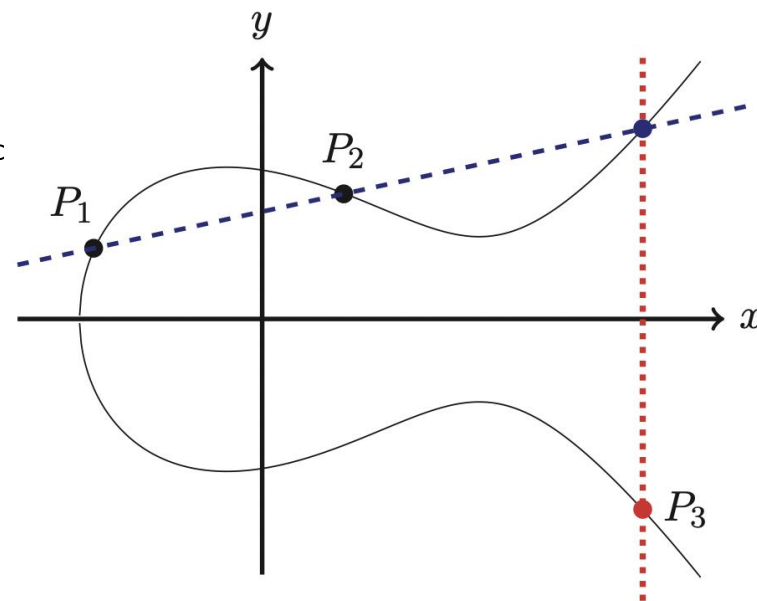
$$E: y^2 = x^3 - x + 1$$

- 如果 P_1 和 P_2 在E上, 我们可以定义加法,

$$P_3 = P_1 + P_2$$

如图所示

- 我们只需要加法.....



椭圆曲线上的点

- 考虑 $y^2 = x^3 + 2x + 3 \pmod{5}$

表 4.1 曲线 $y^2 = x^3 + 2x + 3 \pmod{5}$ 上的点

$x = 0$	$\Rightarrow$	$y^2 = 3 \Rightarrow$ 无解 mod 5
$x = 1$	$\Rightarrow$	$y^2 = 6 = 1 \Rightarrow y = 1, 4 \pmod{5}$
$x = 2$	$\Rightarrow$	$y^2 = 15 = 0 \Rightarrow y = 0 \pmod{5}$
$x = 3$	$\Rightarrow$	$y^2 = 36 = 1 \Rightarrow y = 1, 4 \pmod{5}$
$x = 4$	$\Rightarrow$	$y^2 = 75 = 0 \Rightarrow y = 0, \pmod{5}$

- 那么，椭圆曲线上的点是

$(1,1)$ $(1,4)$ $(2,0)$ $(3,1)$ $(3,4)$ $(4,0)$ 以及无穷远处的点: ∞

椭圆曲线数学

- 补充: $y^2 = x^3 + ax + b \pmod{p}$

$$P_1 = (x_1, y_1), P_2 = (x_2, y_2)$$

$$P_1 + P_2 = P_3 = (x_3, y_3) \text{ where}$$

$$x_3 = m^2 - x_1 - x_2 \pmod{p}$$

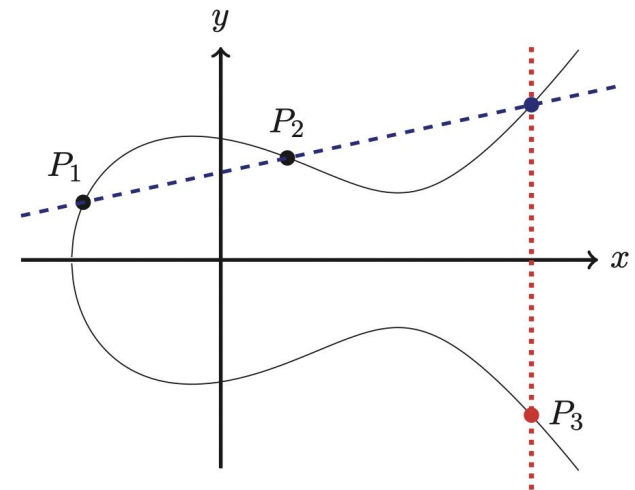
$$y_3 = m(x_1 - x_3) - y_1 \pmod{p}$$

$$\text{And } m = (y_2 - y_1) * (x_2 - x_1)^{-1} \pmod{p}, \text{ if } P_1 \neq P_2$$

$$m = (3x_1^2 + a) * (2y_1)^{-1} \pmod{p}, \text{ if } P_1 = P_2$$

Special cases: If m is infinite, $P_3 = \infty$, and

$$\infty + P = P \text{ for all } P$$



椭圆曲线加法

- 考虑函数 $y^2 = x^3 + 2x + 3 \pmod{5}$ 。曲线上的点为 $(1,1)$ 、 $(1,4)$ 、 $(2,0)$ 、 $(3,1)$ 、 $(3,4)$ 、 $(4,0)$ 和 ∞ 。

- $(1,4) + (3,1) = P_3 = (x_3, y_3)$ 是什么?

$$m = (1-4) \cdot (3-1)^{-1} = -3 \cdot 2^{-1} = 2(3) = 6 = 1 \pmod{5}$$

$$x_3 = 1 - 1 - 3 = 2 \pmod{5}$$

$$y_3 = 1(1-2) - 4 = 0 \pmod{5}$$

- 在这条曲线上, $(1,4) + (3,1) = (2,0)$

ECC Diffie-Hellman

- 公钥: 椭圆曲线和曲线上的点(x, y)
- 私钥: Alice 的 A 和 Bob 的 B



爱丽丝, A

$A(x, y)$

$B(x, y)$



鲍勃, B

- ❑ Alice 计算 $A(B(x, y))$
- ❑ Bob 计算 $B(A(x, y))$
- ❑ $AB = BA$, 所以它们是相同的。

ECC Diffie-Hellman

- **公钥:** 曲线 $y^2 = x^3 + 7x + b \pmod{37}$ 和点 $(2,5) \Rightarrow b = 3$
- **私钥:** Alice: $A = 4$, Bob: $B = 7$
- Alice 发送给 Bob: $4(2,5) = (2,5) + (2,5) + (2,5) + (2,5) = (7,32)$
- Bob 发送给 Alice: $7(2,5) = (18,35)$
- Alice 计算: $4(18,35) = (22,1)$
- Bob 计算得出: $7(7,32) = (22,1)$

更大的 ECC 示例

- Certicom ECCp-109示例
 - 挑战性问题, 于 2002 年解决
- 曲线E: $y^2 = x^3 + ax + b \pmod{p}$
- 在哪里

$p = 564538252084441556247016902735257$

$a = 321094768129147601892514872825668$

$b = 430782315140218274262276694323197$

- 接下来怎么办?

ECC 示例

- 下一点P位于曲线E上

**$(x,y) = (97339010987059066523156133908935 ,$
 $149670372846169285760682371978898)$**

- 令 $k = 281183840311601949668207954530684$

私钥

- kP 由下式给出

**$(x,y) = (44646769697405861057630861884284,$
 $522968098895785888047540374779097)$**

密文

- 点 kP 也位于曲线E上。

真是个大数字！

- 数字很大，但还不够大。
 - ECCp-109 位（32 位）于 2002 年破解
- 今天，ECC DH需要更多的人。
- 但RSA需要**很多**更大的数字
 - 目前最小的RSA模数为1024位
 - 也就是说，超过300位十进制数字
 - 这大约是ECC示例大小的10倍
 - 2048 位 RSA 模数很常见.....

公钥加密的用途

- 保密性
 - 通过不安全信道传输数据
 - 在不安全的介质上进行安全存储
- 身份验证协议（稍后）
- 数字签名
 - 提供完整性和**不可否认性**
 - 使用对称密钥时不存在不可否认性。

不可否认性

- 爱丽丝向鲍勃订购了100股股票。
- Alice使用对称密钥计算MAC
- 股票下跌，爱丽丝声称她**没有**订购
- 鲍勃能证明是爱丽丝下的订单吗？
- **不！** 鲍勃也知道对称密钥，所以他也可能伪造了MAC码。
- **问题：** 鲍勃知道是爱丽丝下的订单，但他无法证明！

不可否认性

- 爱丽丝向鲍勃订购了100股股票。
- 爱丽丝用她的私钥**签署订单**
- 股票下跌，爱丽丝声称她没有订购
- 鲍勃能证明是爱丽丝下的订单吗？
- **是的！** 爱丽丝用她的私钥签署了订单——只有爱丽丝知道她的私钥！
 - 当然，我们假设Alice的私钥没有丢失/被盗。

公钥表示法

- 使用 Alice 的私钥对消息M进行签名： $[M]_{\text{Alice}}$
- 使用 Alice 的公钥加密消息M： $\{M\}_{\text{Alice}}$
- 然后

$$\{[M]_{\text{Alice}}\}_{\text{Alice}} = M$$

$$[[M]_{\text{Alice}}]_{\text{Alice}} = M$$

签名加密顺序

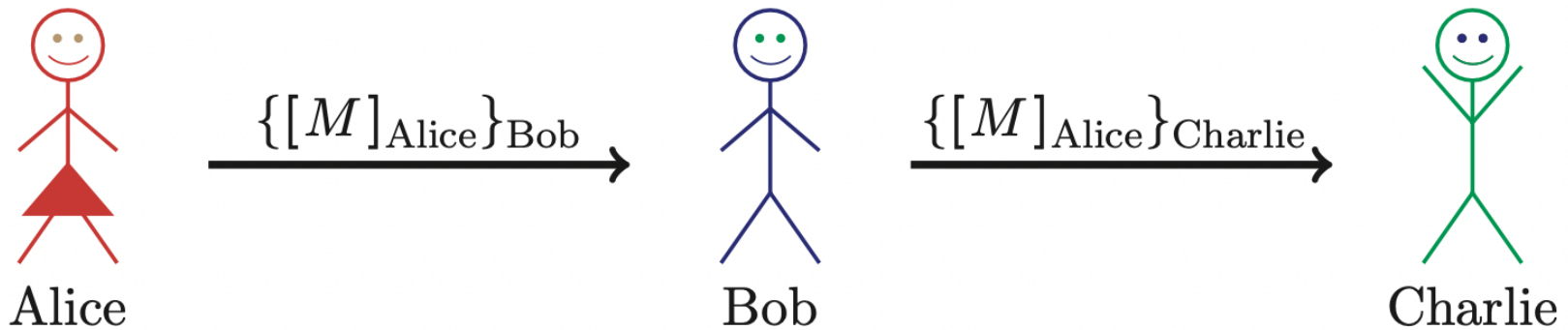
- 签名和加密 vs 加密和签名

保密性和不可否认性

- 假设我们想要保密性和完整性/不可否认性
- 公钥加密能否同时实现这两个目标?
- 爱丽丝给鲍勃发送消息M, 内容是.....
 - **签名和加密:** $\{[M]_{\text{爱丽丝}}\}_{\text{鲍勃}}$
 - **加密和签名:** $[\{M\}_{\text{Bob}}]_{\text{爱丽丝}}$
- 顺序会有影响吗?

签名和加密

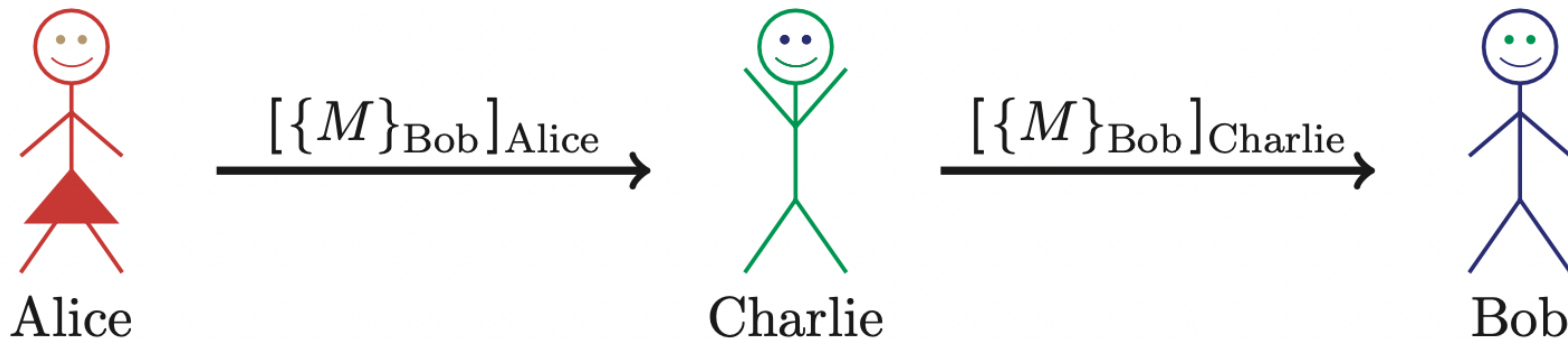
- $M = \text{“我爱你”}$



- ❑ **问：** 出了什么问题？
- ❑ **A：** 没问题——公钥是公开的

加密和签名

- M = “我的理论，这是我自己的理论……”



- 注意，查理无法解密 M
- 问：问题是什么？
- A：没问题——公钥是公开的

公钥基础设施 (PKI)

- 公钥基础设施 (PKI) 是安全地使用公钥密码所需要的所有事务的统称。
- 将PKI的各个部分组合在一起实现该功能是很复杂的。

公钥基础设施(PKI)

- 公钥基础设施 (PKI): 安全使用公钥加密所需的一切
 - 密钥生成与管理
 - 证书颁发机构 (CA) 或证书颁发机构
 - 证书吊销列表 (CRL) 等。
- PKI 没有统一的标准
- 我们提到了3种通用的“信任模型”。
 - 我们只讨论 CA (或CAs) 。

公钥证书

- 数字**证书**包含用户名和用户的公钥（可能还包含其他信息）。
- 它由颁发机构（证书颁发机构，CA）签发。
- 当前最大商业证书系统VeriSign和 Let's Encrypt。

$M = (\text{Alice}, \text{Alice 的公钥}), \quad S = [M]_{CA}$

爱丽丝的证书 = (M, S)

- CA的公钥进行验证：

必须验证 $M = \{S\}_{CA}$

证书颁发机构

- 证书颁发机构（CA）是受信任的第三方（TTP），负责创建和签署证书。
- 使用该CA公钥来验证签名
 - 这并**不能告诉我们**证书发送者的身份，因为证书是公开的！
- 如果证书机构犯错，后果很严重。
 - CA 曾经给别人颁发过微软证书！



PKI信任模型 (1)

- 垄断模型 (Monopoly model)
 - 已知宇宙中一个普遍受信任的组织是CA。
 - 如果CA系统遭到入侵，将会造成严重问题。
 - 谁将担任签证颁发机构？政府？
 - 如果你不信任证书颁发机构，那么这个系统对你来说就毫无用处！

PKI信任模型 (2)

- 寡头模式 (Oligarchy)
 - 多个 (“几个”) 受信任的证书颁发机构
 - 目前浏览器都采用了这种方法。
 - 目前存在超过80家公共证书颁发机构 (CA)，但排名前五的机构占据了超过99%的市场份额。
 - 用户可以自行决定信任哪些或哪些证书颁发机构 (CA)

PKI信任模型 (3)

- 无政府模式 (Anarchy model)
 - 人人都是CA.....
 - 用户必须决定信任谁。
 - PGP 中使用的这种方法是：“信任网络”。
- 为什么说是无政府状态？
 - 假设证书由弗兰克签署，而你不认识弗兰克，但你信任鲍勃，鲍勃说爱丽丝值得信赖，爱丽丝也为弗兰克作证。你应该接受这份证书吗？
- 许多其他信任模型/PKI问题

对称密钥与公钥

- 对称密钥
 - 速度快
 - 无需公钥基础设施 (PKI), 但必须生成/分发密钥
- 公钥
 - 签名 (不可否认性)
 - 没有**共享** 保密, 但必须获取正确的用户密钥 (CA、TTP、PKI 等)。

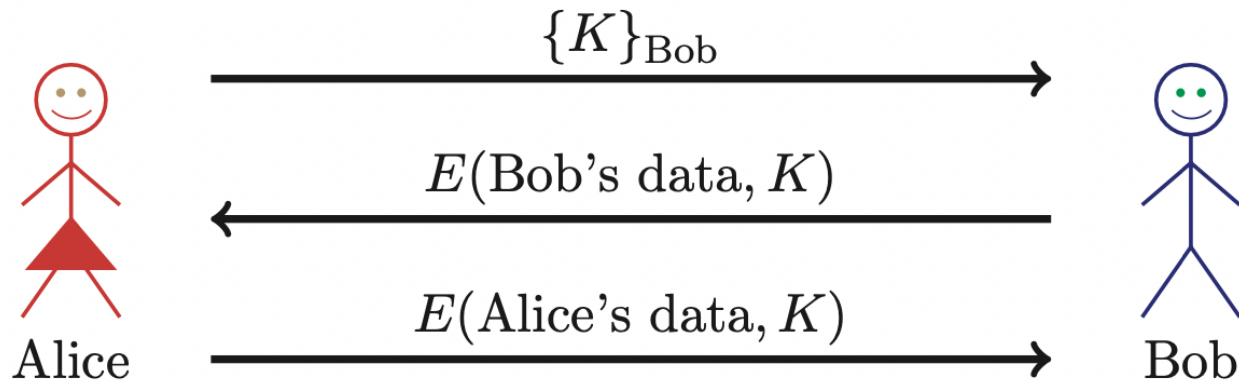
符号提示

- 公钥表示法
 - 使用 Alice 的私钥签名M
 $[M]_{\text{爱丽丝}}$
 - 使用 Alice 的公钥加密M
 $\{M\}_{\text{爱丽丝}}$
- 对称密钥表示法
 - 使用对称密钥加密P K
 $C = E(P, K)$
 - 使用对称密钥解密C语言 K
 $P = D(C, K)$

现实世界的保密性

• 混合密码方式

- 公钥加密用于建立密钥
- 对称密钥加密用于加密数据.....



□ 鲍勃能确定他是在和爱丽丝说话吗？

第五章： Hash 函数

哈希函数动机

- 假设爱丽丝加密了密文M
 - Alice 向 Bob 发送M和 $S = [M]_{\text{Alice}}$ 给 Bob
 - Bob 验证 $M = \{S\}_{\text{Alice}}$
 - Alice可以直接发送S吗?
- 如果M很大, **计算**和**发送** $[M]_{\text{Alice}}$ 的成本就很高。
- 假设爱丽丝对 $h(M)$ 进行签名, 其中 $h(M)$ 是M的一个更小的指纹。
 - Alice 向 Bob 发送M和 $S = [h(M)]_{\text{Alice}}$ 给 Bob
 - Bob 验证 $h(M) = \{S\}_{\text{Alice}}$

哈希函数动机

- 所以, Alice 计算了 $h(M)$
 - 也就是说, Alice 计算 $S = [h(M)]_{\text{Alice}}$
 - 然后爱丽丝发送 (M, S) 致鲍勃
 - Bob 验证 $h(M) = \{S\}_{\text{Alice}}$
- $h(M)$ 必须满足哪些性质?
 - 假设 Trudy 找到 M' , 使得 $h(M) = h(M')$ 。
 - 然后 Trudy 可以将 (M, S) 替换为 (M', S) 。
- Bob 能检测到这种篡改吗?
 - 不, 因为 $h(M') = h(M) = \{S\}_{\text{Alice}}$

加密哈希函数

- 加密哈希函数 $h(x)$ 必须提供
 - **压缩** 输出长度较小
 - **效率** —— 对于任意 x ,
 - **单程** 给定一个值 y , 不可能找到一个 x 使得 $h(x) = y$
 - **弱抗碰撞性** 已知 x 和 $h(x)$, 无法求得 $y \neq x$ 使得 $h(y) = h(x)$
 - **强抗碰撞性** —— 无法找到 **任何** x 和 y , 其中 $x \neq y$, 且满足 $h(x) = h(y)$
- 碰撞事故很多, 但很难找到 **任何一起**。

生日问题1

- 假设房间里有N个人.....
- N必须有多大，我们才能预期 找到一个和我同一天生日的人？
 - 期望意味着概率 $\geq 1/2$
 - 解方程： $1/2 = 1 - (364/365)^N$ ，求N
 - 我们发现 $N = 253$

生日问题2

- 房间里至少要有多少人，才能找到两个或两个以上同一天生日的人？
 - 再次强调，期望概率 $\geq 1/2$
 - $1 - 365/365 \cdot 364/365 \cdots (365 - N + 1)/365$
 - 令 $N = 1/2$ 并求解： **$N = 23$**
- 令人惊讶？ 自相矛盾？
- 或许并非如此：“应该”大约是 $\sqrt{365}$ ，因为我们比较的是所有**配对**。 x 和 y
 - 生日有365种可能。

哈希与生日

- 如果 $h(x)$ 是 N 位，那么可能有 2^N 个不同的哈希值。
- 因此，如果你对 $\sqrt{2^N} = 2^{N/2}$ 个值进行哈希运算，那么你可能会发现冲突。
- **这意味着什么？**“穷举搜索”攻击.....
 - 安全的 N 位哈希需要 $2^{N/2}$ 个工作量才能“破解”。
 - 回想一下，安全的 N 位对称密码的工作因子为 2^{N-1}
- 哈希输出长度VS密文密钥长度的关系？

非加密哈希 (1)

- 数据 $X = (X_1, X_2, X_3, \dots, X_n)$, 每个 X_i 都是一个字节
- 定义 $h(X) = (X_1 + X_2 + X_3 + \dots + X_n) \bmod 256$
- 这是一个安全的加密哈希吗?
- 例如: $X = (10101010, 00001111)$
- 哈希值为 $h(X) = 10111001$
- 如果 $Y = (00001111, 10101010)$, 则 $h(X) = h(Y)$
- 碰撞很容易发现, 所以**不是**安全的...

非加密哈希 (2)

- 数据 $X = (X_1, X_2, X_3, \dots, X_n)$

- 假设哈希定义为

$$h(X) = (nX_1 + (n-1)X_2 + (n-2)X_3 + \dots + 2 \cdot X_{n-1} + X_n) \bmod 256$$

- 这是一个安全的加密哈希吗?
- 注意: $h(10101010, 00001111) \neq h(00001111, 10101010)$
- 但是 $h(00000001, 00001111) = h(00000000, 00010001)$
- 虽然不“安全”，但这个哈希值被用于（非加密的）应用程序 rsync中。

非加密哈希 (3)

- 循环冗余校验 (CRC)
- 本质上，CRC 是长除法运算中的余数。
- 适用于检测突发**错误**
 - 这种随机误差不太可能导致碰撞。
- 但**很容易建造**碰撞
 - 在加密货币领域，Trudy 是敌人，而不是“随机数”。
- 在需要进行加密完整性检查的地方（例如 WEP），CRC 被错误地使用了。

流行的哈希函数

- **MD5** —Message Digest 消息摘要
 - 128 位输出
 - MD5碰撞很容易找到，所以它有问题。
- **SHA-1** 美国政府标准，其内部运作方式与 MD5 类似
 - 180 位输出
- MD5与SHA-1 的hash函数由若干轮组成，同分组密码相似。
- Tiger hash函数比MD5与SHA-1 更复杂。

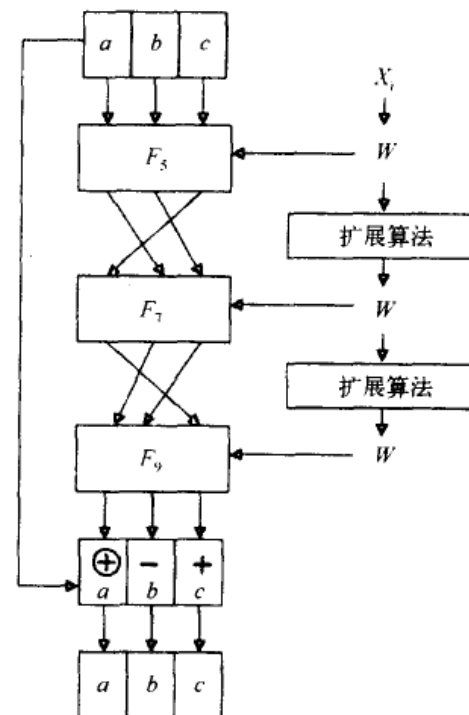
Tiger hash函数

- 输入：512位，或者输入长度填充为512位的整数倍。
- 输出：192位
- 目的：适应64位处理器。
- 使用了四个S盒，受到之前分组密码的影响。
- a,b,c 都是64位。

$a = 0x0123456789ABCDEF$

$b = 0xFEDCBA9876543210$

$c = 0xF096A5B4C3B2E187$



加密哈希设计

- 所需特性：雪崩效应
 - 输入位改为 1 位应该会影响大约一半的输出位。
- 加密哈希函数由若干轮组成。
- 既要安全又要速度
 - 几轮之后出现“雪崩效应”
 - 但简单的回合
- 与分组密码的设计类似

HMAC

- 保护消息的完整性?
 - 使用分组密码密文链接 (CBC) 模式计算MAC
 - **计算hash的MAC, 即HMAC**
- HMAC是一种密钥加密哈希
 - 我们为什么需要钥匙?
- 如何计算HMAC ?
- 两个显而易见的选择: $h(K, M)$ 和 $h(M, K)$
- 哪个更好?

HMAC

- 我们应该将HMAC计算为 $h(K, M)$ 吗?
- 以块为单位计算的哈希值
 - $h(B_1, B_2) = F(F(A, B_1), B_2)$, 其中 F 为某个函数, A 为常数。
 - 那么 $h(B_1, B_2) = F(h(B_1), B_2)$
- 令 $M' = (M, X)$
 - 那么 $h(K, M') = F(h(K, M), X)$
 - 无需 K , 攻击者就可以计算 M' 的HMAC。
- $h(M, K)$ 更好吗?
 - 是的, 但是.....如果 $h(M') = h(M)$, 那么我们可能有 $h(M, K) = F(h(M), K) = F(h(M'), K) = h(M', K)$ 。

HMAC的正确方法

- RFC 2104 中有详细描述
- 设B为哈希表的块长度（以字节为单位）
 - MD5 和 SHA-1 的B 值均为 64
- ipad = 0x36重复B次
- opad = 0x5C重复B次
- 然后

$$\text{HMAC}(M,K) = h(K \oplus \text{opad}, h(K \oplus \text{ipad}, M))$$

哈希用途

- 身份验证协议
- 消息完整性 (HMAC)
- 消息指纹
- 数据损坏检测
- 数字签名效率
- 此外，还有许许多多巧妙/令人意想不到的用途.....
 - 包括区块链和加密货币

在线投标

- 假设爱丽丝、鲍勃和查理是竞标者
- 爱丽丝计划竞标A、鲍勃B和查理C。
- 他们不相信投标会保密。
- 可能的解决方案是什么？
 1. Alice、Bob 和 Charlie 提交**哈希值**。 $h(A)$, $h(B)$, $h(C)$
 2. 所有已接收并发布到网上的哈希值
 3. 随后，投标方A、B和C提交并公布了投标结果。
- 哈希值不会泄露出价（单向）。
- 哈希值发送后无法更改出价（冲突）
- 但这里面存在一个严重的缺陷.....

垃圾邮件清理

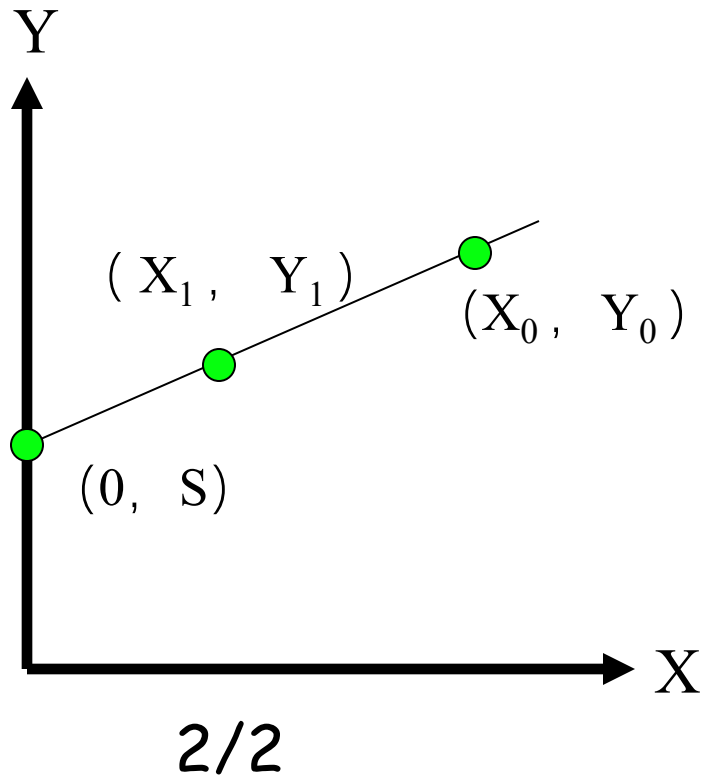
- M: e-mail
- T: 当前时间,
- 消息M的发送者必须找到值R使得

$$h(M, R, T) = (\underbrace{00 \dots 0}_N, X)$$

- 发送者必须找到一个值R使得上式输出的hash值的前N位都是0.
- 发送者计算量: 平均做 2^N 次hash计算。
- 接收者计算量: 一次hash计算

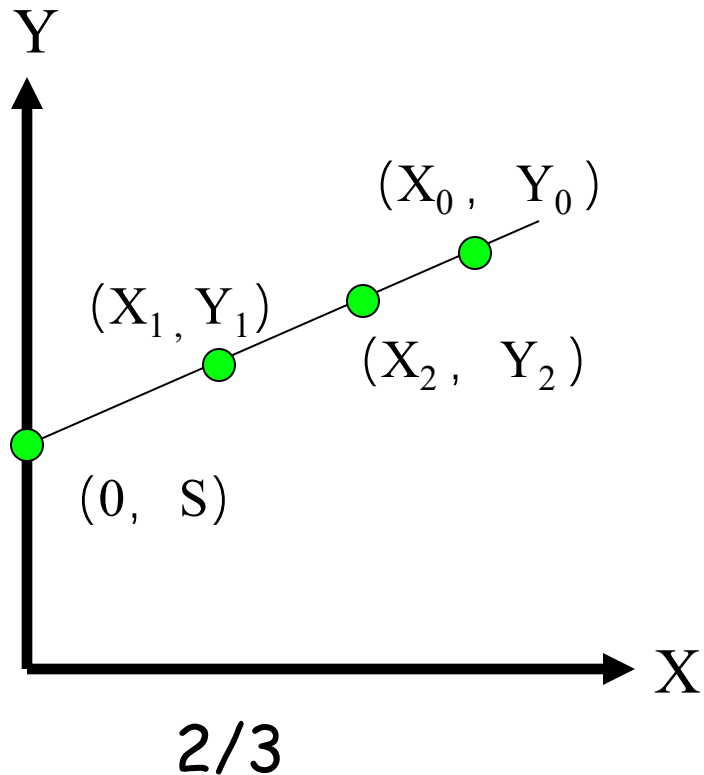
秘密分享

Shamir的秘密分享



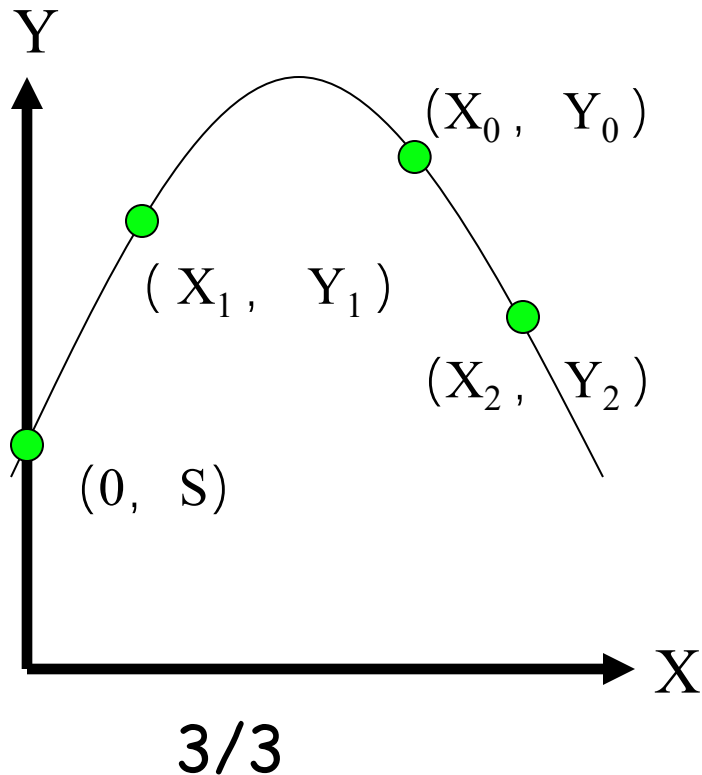
- ❑ 两点确定一条直线
- ❑ 将 (X_0, Y_0) 赋给Alice
- ❑ 将 (X_1, Y_1) 赋值给 Bob
- ❑ 然后，爱丽丝和鲍勃必须合作找到秘密S
- ❑ 也适用于离散情况
- ❑ 对于任意 m ，都很容易构造“ m 取 n ”方案。 $\leq n$

Shamir的秘密分享



- ❑ 将 (X_0, Y_0) 赋给Alice
- ❑ 将 (X_1, Y_1) 赋值给 Bob
- ❑ 将 (X_2, Y_2) 赋给查理
- ❑ 然后任意**两个**可以合作寻找秘密S
- ❑ 没有人可以确定S
- ❑ “三分之二”方案

Shamir的秘密分享

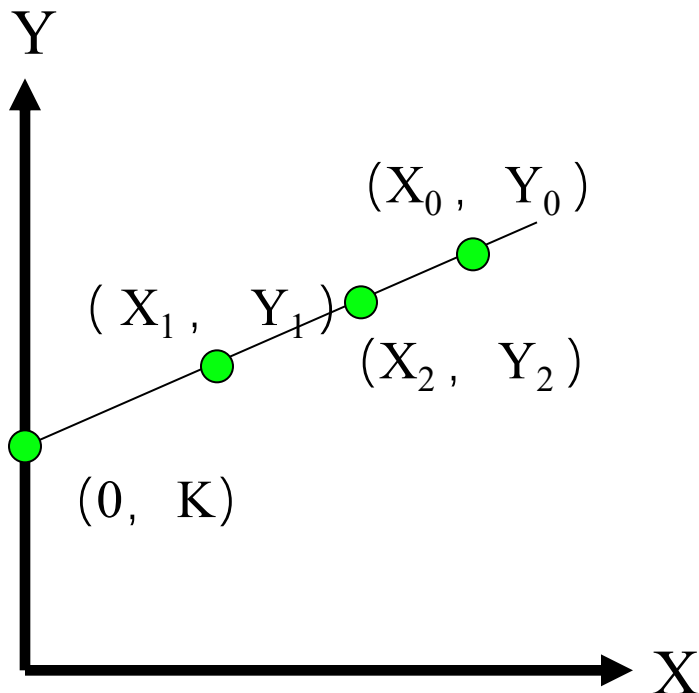


- ❑ 将 (X_0, Y_0) 赋给Alice
- ❑ 将 (X_1, Y_1) 赋值给 Bob
- ❑ 将 (X_2, Y_2) 赋给Charlie
- ❑ 3 分确定抛物线
- ❑ 爱丽丝、鲍勃和查理必须合作才能找到S
- ❑ “三选三”方案
- ❑ “四选三”怎么样?

秘密共享功能是否可用?

- **钥匙托管** 假设你的密钥必须存储在某个地方。
- 密钥可以通过法院命令“追回”。
- 但你不会信任联邦调查局来保管你的钥匙。
- 我们可以使用秘密共享
 - 比如说, 三个不同的政府机构
 - 两人必须合作才能找回钥匙

秘密共享文案



- ❑ 你的对称密钥是K
- ❑ 将 (X_0, Y_0) 指向 FBI
- ❑ 将 (X_1, Y_1) 点指向 DoJ
- ❑ 将点 (X_2, Y_2) 指向 DoC
- ❑ 要找回您的密钥K，三个机构中必须有**两个机构合作**。
- ❑ 任何一个机构都无法获得K

视觉密码学

- 另一种形式的秘密分享.....
- 爱丽丝和鲍勃“分享”了一张照片
- 两人必须合作才能揭示图像
- 没有人能从爱丽丝或鲍勃的那部分照片中了解到任何关于图像的信息。
 - 也就是说，两部分都是必需的。
- 这有可能吗？

视觉密码学

- 如何“分享”像素？
- 假设图像是黑白的
 - 那么每个像素要么是黑色，要么是白色。
 - 我们按所示方式分割像素

	Pixel	Share 1	Share 2	Overlay
a.				
b.				
c.				
d.				

分享黑白图片

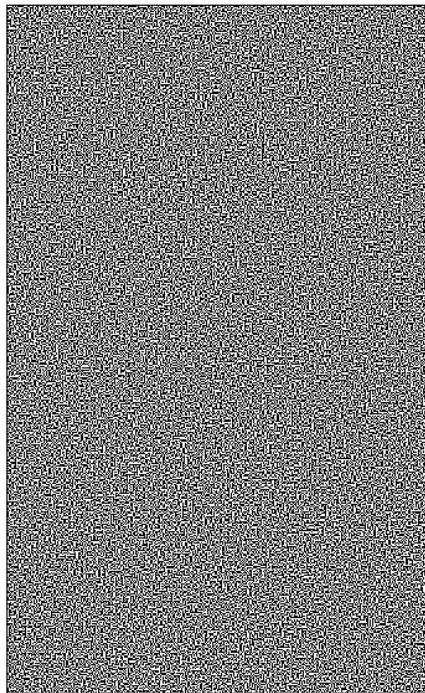
- 如果像素为白色，则随机选择a或b作为 Alice/Bob 的份额。

- 如果像素为黑色，则随机选择c或d。
- 一份份额中**没有任何信息**

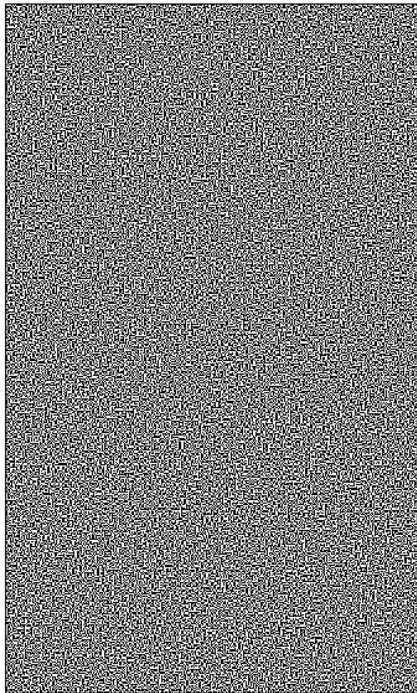
	Pixel	Share 1	Share 2	Overlay
a.				
b.				
c.				
d.				

可视化加密示例

- 爱丽丝的份额



- 鲍勃的份额



- 叠加之后



视觉加密

- 视觉“加密”与传统加密有何异同?
- 在视觉加密中，没有密钥.....
 - 或者，也许这两张图片都是关键?
- 利用加密技术进行穷举搜索
 - 除了一次一密
- 对视觉加密进行全面搜索?
 - 不可能进行穷举搜索!

视觉加密

- 可视化加密——无需穷举搜索.....
- 视觉加密与传统加密有何异同？
 - 视觉加密在*信息论上是安全的*。——秘密共享方案也同样如此
 - 常规加密的目标是使密码分析在计算上不可行。
- **秘密共享**
 - 从通常意义上讲，它并不是密码学中的一种。

随机数

- 用于生成**密钥的随机数**
 - 对称键
 - RSA: 质数
 - 迪菲-赫尔曼: 秘密指数a和b
- 用于网络安全协议的随机数
 - 有时这样的顺序是可以的
 - 有时 nonce 必须是随机的
- 随机数也用于模拟、统计等领域。
 - 此类应用需要“统计学意义上的”随机数。

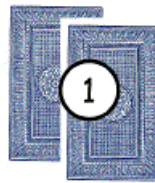
随机数

- **加密随机数**必须是统计上随机且**不可预测的**。
- 假设服务器生成对称密钥
 - 爱丽丝: K_A
 - 鲍勃: K_B
 - 查理: K_C
 - 戴夫: K_D
- 爱丽丝、鲍勃和查理都不喜欢戴夫.....
- 爱丽丝、鲍勃和查理三人合作, 一定无法确定 K_D 的值。

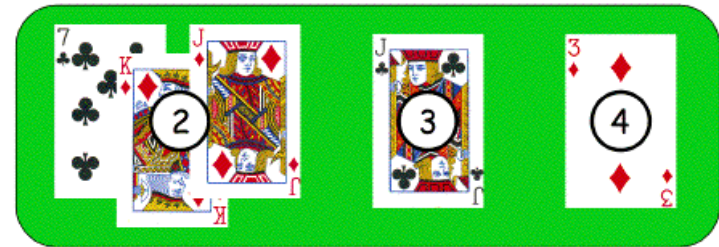
非随机随机数

- 用于洗牌的随机数
- 程序没有生成随机洗牌。
- 这是个严重的问题吗？

- 在线版德州扑克
 - ASF软件公司



Player's hand



Community cards in center of the table

洗牌

- 共有 $52! > 2^{225}$ 种可能的洗牌组合
- 该扑克程序使用“随机”的 32 位整数来决定洗牌顺序。
 - 因此, 只有 2^{32} 种不同的洗牌方式可能发生。
- 代码中使用了 Pascal 伪随机数生成器 (PRNG):
Randomize()
- 伪随机数生成器的种子值是自午夜以来的毫秒数的函数。
- 一天不到 2.27 毫秒
 - 可能的组合少于 2^{27} 种。

洗牌

- 基于午夜以来的毫秒数的种子
- 每次洗牌都会重新播种伪随机数生成器。
- 通过与服务器同步时钟，需要测试的洗牌次数 $< 2^{18}$
- 实时测试所有 2^{18} 个
 - 用“明牌”测试每种可能的洗牌方式。
- 攻击者在第一轮下注后的五轮下注中就知道了**所有牌！**

扑克示例

- 扑克程序就是一个极端例子
 - 但常见的伪随机数生成器是可预测的。
 - 问题只在于，在确定序列之前，需要观察多少个输出。
- 加密随机序列不可预测
 - 例如，RC4 密码的密钥流
 - 但是“种子”（或密钥）的选择仍然是个问题！
- 如何生成初始**随机**值？
 - 键（以及在某些情况下，种子值）

什么是随机性?

- 真正的“随机”很难定义。
- **熵**是衡量随机性的指标。
- 真正随机性的良好来源
 - 放射性衰变——但是，放射性计算机并不太受欢迎。
 - 物理热噪音硬件设备——市面上有很多不错的选择。
 - 依赖于混沌产生的随机。

随机性

- 软件随机性来源
 - 软件应该是确定性的。
 - 因此，必须依赖外部“随机”事件。
 - 鼠标移动、键盘操作、网络活动等等。
- 通过这些方法可以获得**高质量的随机比特**。
- 但比特的**数量**非常有限。
- 结论是：“使用伪随机过程生成秘密数据会导致伪安全”。

信息隐藏

- 数字水印
 - 示例：向数据中添加“不可见”信息
 - 打击音乐/软件盗版
- 隐写术
 - “秘密”通信渠道
 - 类似于秘密频道（稍后详述）
 - 示例：将数据隐藏在图像文件中

水印

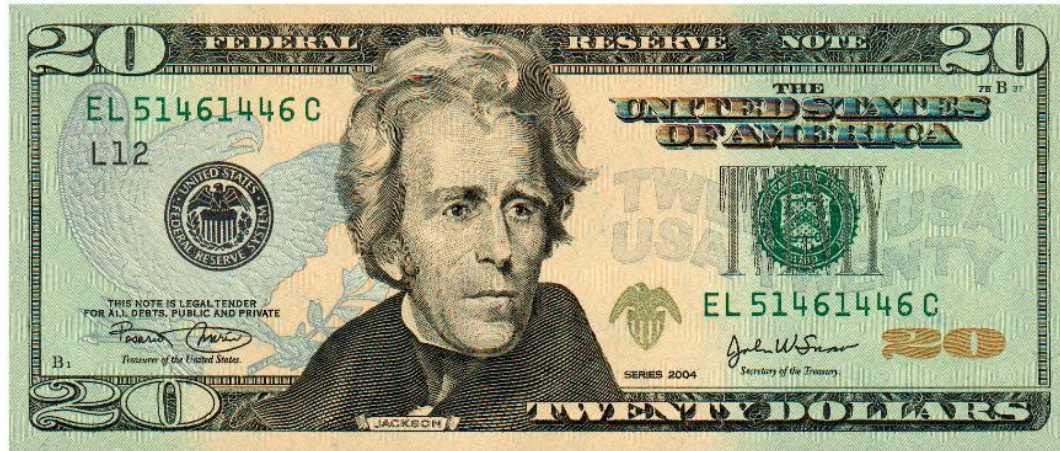
- 给数据添加“标记”
- 水印的可见性（或不可见性）
 - 不可见——水印不明显,在媒体中不能被感知
 - 可见的——例如文档上盖上**绝密**字样
- 水印的强度（或强度不足）
 - 鲁棒水印——即使遭受攻击也能读取
 - 脆弱水印——一旦发生改动则立即消失或被破坏

水印示例

- 添加**强大的无形**标记
 - 如果盗版音乐出现在互联网上，可以追溯到泄露的原始源头。
- 给音频文件添加**脆弱的隐形**标记
 - 如果水印无法辨认，接收者就知道音频已被篡改（完整性）。
- 有时会使用几种类型的组合。
 - 例如，可见水印和强大的隐形水印

水印示例 (1)

- 非数字水印：美元



- ❑ 图像嵌入在纸张的右侧
 - 将账单对着光看，即可看到嵌入的信息

水印示例 (2)

- 给照片添加**隐形水印**
- 有人声称, 1英寸²包含足够的信息来重建整张照片。
- 如果照片损坏, 可以使用水印进行修复!

隐写术

- 据希罗多德（公元前440年希腊）记载
 - 被剃光的奴隶的头
 - 在头上写了信息
 - 让头发重新长出来
 - 派奴隶去传递消息
 - 剃光奴隶的头，露出警告波斯入侵的信息。
- 历史上， 军事领域使用隐写术的频率高于密码学。

图像和隐写术

- 图像通常使用24位表示颜色： RGB

- 红色8位, 绿色8位, 蓝色8位

- 例如

- 0x7E 0x52 0x90是这种颜色吗?

- 0xFE 0x52 0x90是这种颜色吗?

- 尽管

- 0xAB 0x33 0xF0是这种颜色吗?

- 0xAB 0x33 0xF1是这种颜色吗?

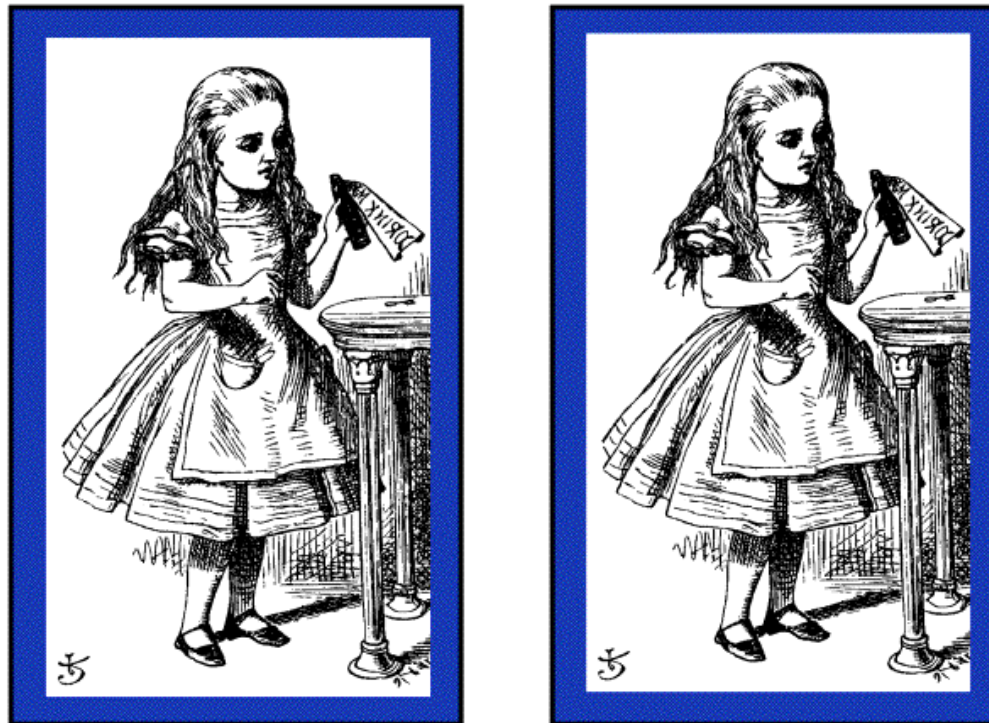
- 低位无关紧要.....

https://www.rapidtables.com/web/color/RGB_Color.html

图像和隐写术

- 给定一个未压缩的图像文件.....
 - 例如, BMP格式
- 我们可以将信息插入到低阶RGB位中
- 由于低阶 RGB 位无关紧要, 因此人眼无法察觉这些变化。
 - 但是计算机程序可以“看到”这些比特。

隐写术示例 1



- 左侧：纯爱丽丝图像
- 右侧：爱丽丝，以及隐藏在图片中的整本《爱丽丝梦游仙境》（pdf）。

隐写术示例

□ 在网页浏览器中打开 `stegoWalrus.html`

"The time has come," the Walrus said,
"To talk of many things:
Of shoes and ships and sealing wax
Of cabbages and kings
And why the sea is boiling hot
And whether pigs have wings."

• “查看源代码”显示：

```
<font color=#000101>“时候到了，”海象说。</font><br>  
<font color=#000100>“要谈论很多事情：</font><br>  
<font color=#010000>关于鞋子、船只和封蜡</font><br>  
<font color=#010000>关于卷心菜和国王</font><br>  
<font color=#000000>以及为什么海水会沸腾</font><br>  
<font color=#010001>还有，猪到底有没有翅膀？”</font><br>
```

□ 隐藏信息： **011 010 100 100 000 101**

隐写术

- 某些格式（例如图像文件）比 HTML 更难被**人类**阅读。
 - 但计算机程序很容易“读取”.....
- 很容易将信息隐藏在**不重要的部分**中。
- 很容易损坏不重要信息片段。
- 为了保证**稳健性**，必须使用**重要位**。
 - 但存储的信息绝不能损坏数据。
 - 串谋攻击也是一个令人担忧的问题。
- 稳健的隐写术很棘手！

信息隐藏： 底线

- 隐藏数字信息并非易事
 - “显而易见”的方法并不**稳健**。
 - **Stirmark**：一款可以让图像中的大多数水印变得无法辨认且不会损坏图像的工具。
 - 隐写术/水印是目前的研究热点。
- 如果怀疑存在信息隐瞒行为
 - 攻击者可能能够使信息/水印无法读取。
 - 攻击者如果拥有原始文档（图像、音频等），则可能能够读取其中的信息。

公钥密码

- 背包密码
- RSA算法
- DH算法
- 椭圆曲线密码
- 签名和不可否认性
- 公钥基础设施 (PKI)

Hash函数

- Hash函数 (MD5和SHA-1)
- Tiger算法
- HMAC
- 秘密共享方案
- 随机数
- 信息隐藏 (水印和隐写术)

Next week

- 高级密码分析

Thank you!